

MediCore — Patient-Doctor Appointment Management System

Modern Application Development 2 — Project Report

1. Student Details

- Name: Niharika Pandey
- Roll Number: 23F3000211
- Email: 23f3000211@ds.study.iitm.ac.in
- Course: BS in Data Science and Applications, IIT Madras (Online)
- About Me: I am a data science student interested in full-stack web development and healthcare technology. This project helped me apply what I learned in MAD 2 — REST APIs, background tasks, SPAs, and PWAs — in a real-world setting.

2. Project Details

Project Title: MediCore — Patient-Doctor Appointment Management System

Problem Statement:

Hospitals need efficient systems to manage patients, doctors, appointments, and treatments. Currently, many hospitals use manual registers or disconnected software, which makes it difficult to manage records, avoid scheduling conflicts, and track patient history. You are required to build a **Hospital Management System (HMS)** web application that allows **Admins, Doctors, and Patients** to interact with the system based on their roles.

Approach:

The backend is built with Flask and the frontend with Vue 3, running as separate services that talk to each other via a REST API. There are three user roles — Admin, Doctor, and Patient — each with their own dashboard. The app also uses Redis for caching, Celery for background tasks, Twilio for SMS/WhatsApp alerts, and Google Calendar for booking events.

3. AI / LLM Declaration

I used Claude (Anthropic) to help write some SQLAlchemy model definitions, generate API documentation samples, and clean up code comments. Overall AI usage was around 10–15%. All the actual logic, debugging, and integration were done by me.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Python 3 + Flask + Flask-RESTful	Backend web framework and REST API routing
SQLAlchemy	ORM to work with the database using Python classes
Flask-JWT-Extended	JWT-based login and role-based access control
Flask-CORS	Allows the Vue frontend to talk to the Flask backend
Redis	Caches frequently read data to speed up responses
Celery + Celery Beat	Runs background jobs and scheduled tasks
SQLite	Simple database used during development
Vue 3 (Composition API)	Frontend single page application framework

Technology / Library	Purpose
Vue Router	Handles page navigation on the frontend
Axios	Makes HTTP requests from Vue to the Flask API
Chart.js	Shows charts on the admin dashboard
jsPDF	Lets users download reports as PDF from the browser
Bootstrap 5 + Custom CSS	Makes the UI responsive and mobile-friendly
Twilio API	Sends SMS and WhatsApp appointment reminders
Google Calendar API	Creates calendar events when a booking is made
Service Worker + Web Manifest	Enables PWA install and offline support

5. Database Schema / ER Diagram

Tables:

- Account — stores login info: id, email, password_hash, role (admin / doctor / patient)
- Physician — doctor profile: id, account_id (FK), division_id (FK), name, specialisation, fees
- Client — patient profile: id, account_id (FK), name, dob, contact, blood_type
- Division — medical department lookup: id, name, description
- Booking — appointment record: id, physician_id (FK), client_id (FK), date, status, amount
- CaseNote — medical notes: id, booking_id (FK), physician_id (FK), client_id (FK), notes, date
- PhysicianSchedule — doctor availability: id, physician_id (FK), day_of_week, start_time, end_time

Relationships:

- One-to-One → Account → Physician / Account → Client
- One-to-Many → Division → Physician
- One-to-Many → Physician → Booking, CaseNote, PhysicianSchedule
- One-to-Many → Client → Booking, CaseNote
- One-to-One → Booking → CaseNote

ER Diagram:



6. API Resource Endpoints

All responses return JSON. Login-protected routes need a JWT Bearer token in the Authorization header.

Auth (/api/auth)

Endpoint	Method	Description
POST /login	POST	Log in and get a JWT token
POST /register	POST	Create a new doctor or patient account
GET /me	GET	Get the currently logged-in user's profile

Admin (/api/manage)

Endpoint	Method	Description
GET /overview	GET	Dashboard stats — total doctors, patients, bookings
GET, POST /physicians	GET/POST	List all doctors or add a new one
PUT, DELETE /physicians/<id>	PUT/DEL	Update or remove a doctor
GET /clients, GET /clients/<id>	GET	List patients or view one patient's full profile
GET /bookings	GET	View all bookings with status filter
GET /reports/monthly	GET	Monthly report with booking counts and revenue

Doctor (/api/physician)

Endpoint	Method	Description
GET, PUT /profile	GET/PUT	View or update own profile
GET /bookings	GET	See all appointments, filtered by status
PUT /bookings/<id>/approve complete cancel	PUT	Approve, complete, or cancel a booking
GET /notes, POST /notes/<client_id>	GET/POST	View or create case notes for a patient
GET, POST /schedule	GET/POST	View or set weekly availability

Patient (/api/client)

Endpoint	Method	Description
GET, PUT /profile	GET/PUT	View or update own profile
GET /physicians	GET	Browse available doctors (cached with Redis)
POST /bookings/<physician_id>	POST	Book an appointment — also creates a Google Calendar event
PUT /bookings/<id>/cancel	PUT	Cancel an appointment

Endpoint	Method	Description
POST /payments	POST	Make a demo payment (card or UPI) for a booking
POST /download-csv	POST	Start a background job to export medical history as CSV
GET /task/<task_id>	GET	Check job status and download the CSV when ready

YAML API definition file is included separately in the submission ZIP as *api.yaml*.

7. Architecture and Features

Architecture Overview:

- The backend (Flask) and frontend (Vue 3) run as separate apps and communicate only through the REST API
- Backend folders: /routes (one file per role), /models (database classes), /tasks (Celery jobs)
- Frontend folders: /views (one page per role), /components (shared UI), /router (navigation), /services (API calls)
- Redis caches common responses; Celery handles all async and scheduled work

Default Features:

- JWT login with role-based access — each role only sees what they are allowed to
- Admin can add, edit, or remove doctors and view patient records
- Doctors can set their schedule, approve bookings, and write case notes
- Patients can search for doctors, book appointments, and cancel them
- Redis caches doctor listings and the admin overview to keep the app fast

Additional Features:

- Monthly report — shows total bookings, status breakdown, and revenue; can be downloaded as PDF
- Async CSV export — patient medical history is exported in the background using Celery, so the page doesn't freeze
- Demo payment — patients can pay via card or UPI (validated inputs); a transaction ID is generated and saved
- SMS and WhatsApp reminders — Celery checks every hour for appointments in the next 3 hours and sends Twilio alerts
- Google Calendar — a calendar event with email invite is created automatically when a patient books an appointment
- Daily Google Chat summary — Celery posts a doctor-wise appointment summary to a Google Chat webhook every day
- Monthly physician email reports — each doctor gets an auto-generated HTML report of their monthly activity
- PWA — the app can be installed on desktop or mobile via the browser, and works offline for cached pages
- Fully responsive UI — works on both desktop and mobile using Bootstrap 5

8. Video Presentation

The walkthrough video (under 3 minutes) shows login for all three roles, the booking workflow, the admin monthly report, PWA install, CSV export, and an SMS reminder.

Drive Link: [demovideo](#)

9. Screenshots

I'm attaching the screenshots of the demo Twilio SMS and WhatsApp booking alert along with the Google Calendar reminder message. These are just demo updates further they'll be connected with the exact details

```
=====
WHATSAPP DEMO
=====

Enter phone number (e.g., +918100397809): +8100397809

[WhatsApp] Sending to: whatsapp:+8100397809
-----
[WhatsApp] Connecting to Twilio WhatsApp API...
[WhatsApp] Message queued...
[WhatsApp] Message Content:
Hello! This is a demo WhatsApp message from MediCore. Your appointment is confirmed! 🍕
[WhatsApp] Status: DELIVERED
[WhatsApp] Message SID: SM9876543210fedcba9876543210fedcba
[WhatsApp] Cost: $0.0050

[DEMO] Press Enter to return to menu...
```

```
>>
=====
GOOGLE CALENDAR DEMO
=====

Enter email (e.g., patient@example.com): abc@gmail.com

[Google Calendar] Creating event for: abc@gmail.com
-----
[Calendar] Authenticating with Google...
[Calendar] Building calendar service...
[Calendar] Creating event...

Event Details:
  Title: Medical Appointment - Dr. Dr. Priya Mehta
  Date: December 15, 2024
  Time: 02:00 PM
  Duration: 1 hour
  Location: MediCore Hospital, Main Building
  Attendees: abc@gmail.com

Reminders Set:
  - Email: 3 hours before
  - Popup: 30 minutes before
  - SMS: 3 hours before (via Twilio)

[Calendar] Status: EVENT CREATED
[Calendar] Event ID: evt_abc123xyz789
[Calendar] Event Link: https://calendar.google.com/event?eid=abc123xyz789
[Calendar] Invitation sent to: abc@gmail.com

[DEMO] Press Enter to return to menu...

=====
MEDICORE NOTIFICATION DEMO MENU
```

```
=====
SMS DEMO
=====

Enter phone number (e.g., +918100397809): +8100397809

[SMS] Sending to: +8100397809
-----
[SMS] Connecting to Twilio...
[SMS] Message queued...
[SMS] Message Content:
This is a demo SMS from MediCore. Your appointment is confirmed!
[SMS] Status: DELIVERED
[SMS] Message SID: SM1234567890abcdef1234567890abcdef
[SMS] Cost: $0.0075

[DEMO] Press Enter to return to menu...
```